



RESTHeart AI

Make Your MongoDB Backend AI-Ready

RESTHeart 9.9

The Challenge

AI agents and LLMs need **data** — your data



Semantic Search

Users expect search that understands meaning, not just keywords



AI Agents

LLM agents need to query, read, and update your MongoDB data



RAG Pipelines

Retrieval-Augmented Generation requires chunking and indexing your documents

What RESTHeart AI Delivers

Four capabilities, one optional plugin. **All of it shipped in RESTHeart 9.9 — open source.**



OAuth 2.0

Agents can authenticate



Vector Search

Search by meaning



AI Providers

Bring your own model



MCP Server

Agents can use your API

OAuth 2.0

The prerequisite for everything else

An AI agent cannot use an API it cannot authenticate against. Before MCP, RESTHeart had to speak the standards agents already speak.

It now does: a `/token` endpoint, auto-discovery, and the protected-resource metadata that MCP's own OAuth profile requires.

Any OAuth 2.0 client works, agent or not.

RFC 6749 · RFC 8414 · RFC 9728 · restheart.org/docs/ai/mcp

Vector Search

Search that understands meaning

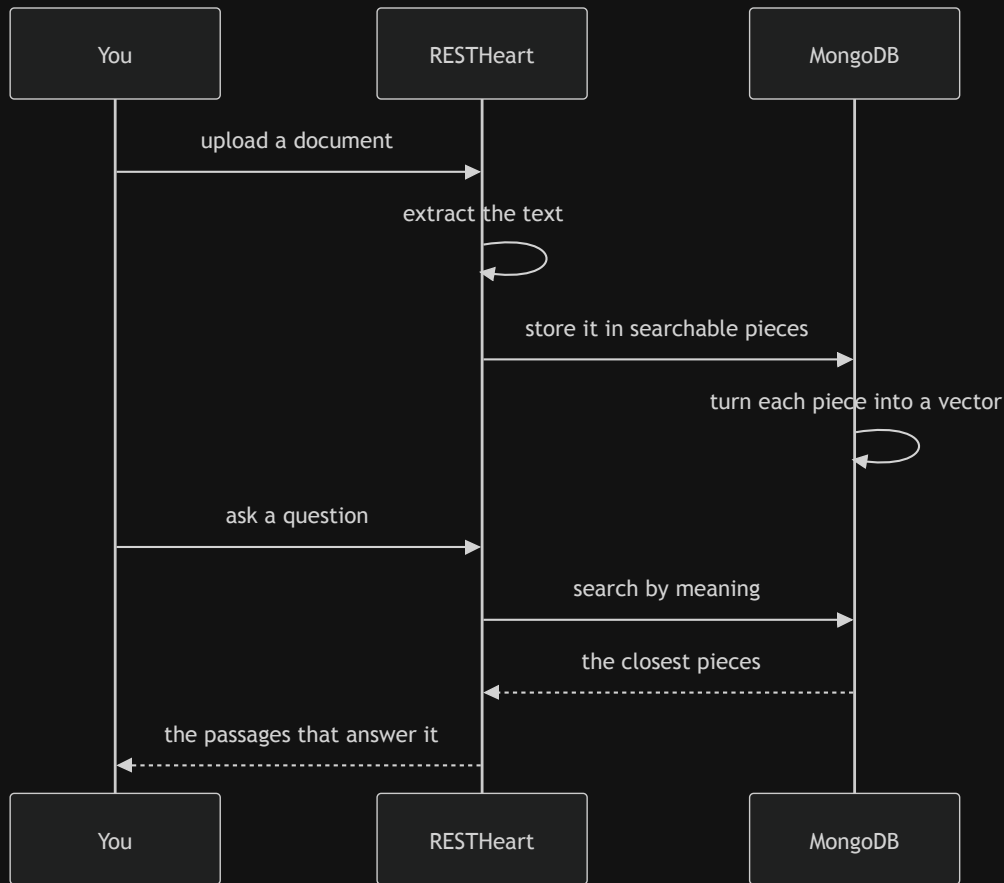
Semantic search over your own data, with no pipeline to build and no second system to run.

- **Indexes** — declare a vector index on any collection
- **Documents** — upload a PDF or a Word file; RESTHeart extracts the text and stores it in searchable pieces
- **Embeddings** — MongoDB generates the vectors, or you bring your own model
- **Reranking** — refine the results before they reach the agent

restheart.org/docs/ai/vector-search

From a PDF to an Answer

Upload a document. Ask a question. Nothing in between is yours to build.



Bring Your Own Model

When MongoDB's own embeddings are not the ones you want

MongoDB can generate the vectors for you. When it cannot — or when you need a particular model — RESTHeart calls the provider itself.

- **Embeddings** — OpenAI, Voyage AI, AWS Bedrock, or Ollama for a model that never leaves your machine
- **Reranking** — Cohere, Voyage AI, AWS Bedrock
- **Wherever you need it** — inside an aggregation, or automatically as documents are written
- **Nothing changes for clients** — the same requests, the same responses

restheart.org/docs/ai/vector-search

The MCP Server

Teach the agent your API, don't rebuild it

RESTHeart already exposes a full REST API. The MCP server does not reimplement it — it teaches the agent how to use it.

Three tools are the whole navigation layer: **what exists**, **how to call it**, and **a credential to call it with**. The agent then makes the request itself, straight to the API.

No data passes through the MCP tools, and there is no second API to keep in step with the first. Your existing endpoints stay the only source of truth.

restheart.org/docs/ai/mcp

Every Collection Is Already There

Nothing to write, nothing to wire.

What an agent gets

Your collections, aggregations, change streams and GraphQL apps — discoverable, callable, and readable straight into its context. It can subscribe to a collection and be told when the data changes.

Your permissions decide what it sees. An agent is a caller like any other.

What you write

For MongoDB and GraphQL: a line of metadata on the collection.

For your own plugin: implement one interface. Every method has a default, so the common case is an annotation and nothing else.

One Deployment, Many Customers

An MCP server that can only serve one customer is a server you have to run once per customer.

RESTHeart runs **one process** and gives every caller a catalogue of its own: an agent discovers its customer's collections, under its customer's own URLs, and nothing else.

Why it is not just an ACL question

Permissions decide what a caller may *read*. They do not stop a catalogue from being built out of everybody's data in the first place — and a list of resource names is already information.

So the catalogue itself is partitioned, not merely filtered.

Agent of customer A

```
list_apis → inventory, orders  
           on a.example.com
```

Agent of customer B, same process

```
list_apis → catalog, invoices  
           on b.example.com
```

Neither can see, name, or read the other's. A single deployment serves both, and neither is aware the other exists.

Where This Stands

It was a plan in three phases. All three shipped in 9.9, so this is the starting point, not the destination.



Agents can get in

Standards-compliant OAuth, including the parts MCP's own profile requires.



Your data is searchable by meaning

Vector search, document ingestion, embeddings from MongoDB or from a model you choose.



Agents can use it

An MCP server over your existing API, with a catalogue per customer on a shared deployment.

Summary

RESTHeart 9.9 makes your MongoDB backend **natively AI-ready**

For your data

- ✓ Search by meaning, on any collection
- ✓ Upload a PDF and have it become searchable
- ✓ No pipeline to build, no second system to run
- ✓ Or bring the model you already trust

For your agents

- ✓ Works with Claude, Cursor, VS Code, any MCP client
- ✓ Your collections, discoverable and readable as they are
- ✓ Your permissions decide what an agent sees
- ✓ One deployment, a catalogue per customer

github.com/SoftInstigate/restheart · restheart.org/docs/ai/mcp